Design and Implementation of Distributed Applications 2025-2026 "dida-meetings" Project Report

Diogo Almada , Miguel Barbosa , Rita Melo
Instituto Superior Técnico
diogo.almada@tecnico.ulisboa.pt
miguel.c.barbosa@tecnico.ulisboa.pt
rita.de.melo@tecnico.ulisboa.pt

Abstract

This report describes the design and implementation of the "dida-meetings" project, a distributed meeting management system based on the Paxos consensus algorithm, developed for the Design and Implementation of Distributed Applications course.

1. Baseline

The project was developed starting from a baseline implementation that provided a simplified version of Paxos using Schedule A. Our work extended this baseline to progressively implement a complete *Phase 1*, Multi-Paxos, Reconfigurable Multi-Paxos (Vertical Paxos) and fast TOPIC operations.

2. Design Overview

In this section, we present the main design decisions and implementation details of our solution. Before describing each component, it is important to note that we implemented the second version of Vertical Paxos (VP2). This choice significantly influenced the structure of our Main Loop and the implementation of *Phase 1*.

2.1. Main Loop

The *MainLoop* class represents the central control flow of the server. It continuously monitors incoming client requests and coordinates the execution of the Paxos protocol across all phases. Inside each replica, when a new command arrives, the *MainLoop* verifies whether the current server is the leader for the server's current ballot and determines if *Phase 1* needs to be executed.

When the server first discovers that it is the new leader for the current configuration, permission for *Phase 2* will not be granted yet. Therefore, the loop initiates *Phase 1* to recover any missing log entries from the previously completed ballot. A ballot is defined as completed only once every *Paxos Log* has been rewritten with such ballot on a quorum of acceptors. Once the logs have been updated to match their write ballot to the proposer's ballot and the console grants activation, the server proceeds to execute *Phase 2*, where the leader can propose client commands to the acceptors of the current configuration, defined by the current ballot.

Each time an instance enters *Phase 2*, the corresponding `RequestRecord` associated with that instance's command ID is moved to the `executing` hashtable. This variable indicates that *Phase 2* has been initiated for that instance within the Proposer. As a result, when the Proposer later invokes `getFirstPending()` to retrieve a new command, it will select a different request, since the previous one has already been moved out of the pending table.

Once a learner reaches a quorum of acceptors and marks an instance as decided, the instance is ready for execution by the dedicated Apply thread. There, each instance with a higher number waits until all lower-numbered instances have been processed. This guaranties that all replicas apply commands in the same total order, preserving consistency across all servers.

Between requests, the *MainLoop* remains idle, waiting for more work or console commands.

2.2. Phase 1

Phase 1 is responsible for synchronizing the state of the new leader with the rest of the acceptors when a new ballot is initiated. In our implementation, this phase was extended from the baseline to support Multi-Paxos and Vertical Paxos

(VP2), allowing the leader to recover all instances from the previously completed ballot before proceeding to *Phase 2*.

To enable this, we modified the *protocol buffers* to include detailed log information in the `PhaseOneReply` message. Each reply now contains a list of `PhaseOneInstance` entries representing all Paxos instances known by an acceptor:

- `repeated PhaseOneInstance logs` — all Paxos instances;
- `bool accepted` — indicates whether the new ballot was accepted;
- `int32 maxballot` — the highest ballot number observed, used when the request was aborted.

The new `PhaseOneInstance` structure was designed to store essential per-instance information such as instance number, server id, request ballot, command ID, and value ballot *(write ballot)*. This enables the leader to reconstruct a new updated log.

On the leader side, *Phase 1* replies processing is handled by the `PhaseOneResponseProcessor` class. This component aggregates replies from all acceptors and determines whether a quorum has been achieved. The processor maintains the following:

- an `updated_phase_one_logs` hash table that merges logs from all acceptors, always keeping the entry with the highest ballot;
- a `num_instances_to_accept` variable to get the total number of instances received from the acceptors' logs;

If any acceptor rejects the ballot, the processor sets the `accepted` flag to false and updates the leader's `maxballot` accordingly, causing the *MainLoop* to abort the completed ballot via `abortCompletedBallot()`, which will be better explained later. Otherwise, the leader proceeds to resend the recovered instances to all acceptors and eventually updates the completed ballot state.

This mechanism ensures that before *Phase 2* begins, a quorum of acceptors has updated its logs with the new leader's ballot, which is a key requirement for the correct operation of Vertical Paxos VP2.

2.3. Phase 2

Phase 2 is responsible for proposing client commands to the quorum of acceptors and achieving consensus on each command. In our implementation, each *Phase 2* instance runs in its own thread. This setup allows multiple commands to run at the same time, effectively creating a Multi-Paxos system. The implementation allows the leader to propose new commands continuously without waiting for previous instances to be decided in consensus.

Responses from acceptors are handled by the `PhaseTwoResponseProcessor`. The processor's `onNext` method serves two purposes:

1. In a typical *Phase 2* scenario, it gathers replies for a client command until enough acceptors respond, deciding if the proposal was accepted or rejected. If any acceptor rejects the proposal, we abort, and the processor updates the leader's ballot number.

2. In the case where a new leader is recovering previous instances from the completed ballot (i.e., before receiving permission from the console to propose new commands), `onNext` also tracks how many *Phase 1* instances have been successfully sent and acknowledged to the quorum. Once all recovered instances have been accounted for, the processor updates the completed ballot with its ballot, waiting for the console to give permission for *Phase 2*. If any acceptor rejects the proposal, we abort, the processor updates the leader's ballot number and aborts the completed ballot procedure.

This dual use ensures that *Phase 2* can both propose new commands and help the new leader update the paxos logs with its ballot on every server, maintaining consistency across replicas while respecting the VP2 activation logic.

2.4. Apply Command Thread

The *Apply* thread is responsible for executing client commands on the replicated state once they have been decided by the Paxos protocol. By running in a separate thread, it decouples the execution of commands from the consensus logic, allowing the *MainLoop* to continue proposing new commands without waiting for previous ones to be applied.

Commands are processed in total order according to their log entry numbers, maintaining a consistent execution sequence. The thread continuously checks the next undecided instance and waits until it becomes decided. Once an instance is decided, the corresponding command is retrieved from the *executing* list and applied to the local replica state.

To ensure synchronization between learners and the apply thread, the learner wakes up the apply thread through the `applyLock.notifyAll()` call whenever a new instance reaches the decided state. This guarantees that no decided command remains unapplied. If the Paxos decision for a command is received before the corresponding client request, the thread waits on the request history until the command is available, keeping total order and correctness.

Besides that, NO-OP commands are used in the logs recovery section of the *Phase 1*. If a recovered instance did not have any value proposed, the new leader sends a NO-OP to *Phase 2*. Then, once the learner reaches a quorum of acceptors, instead of executing a command, the *Apply* thread skips the instance and continues with the next one.

This ensures that all replicas apply commands deterministically and in the same order, while maintaining responsiveness and separation of concerns between the consensus and execution layers.

2.5. Vertical Paxos

As mentioned above, for our implementation, we chose the second version of Vertical Paxos (VP2). VP2 requires coordination with a master entity (in our project, the console) to activate a new leader and to handle completed ballots.

Firstly, the `NewBallotReply` message was extended with a `bool accepted` field to indicate whether the completed ballot was successfully updated or aborted. Additionally, we introduced a new `ActivateRequest` message, which allows the console to explicitly grant or deny permission for the new leader to execute *Phase 2* requests. This mechanism ensures that a leader can only propose new commands after the system has reconciled the state of the completed ballot.

The master service manages both new ballot and activation requests. The following flags are used to control the VP2 logic:

- `activated` – indicates whether the console has already responded to the leader with permission (or denial) to proceed.
- `hasPermissionForPhaseTwo` – determines whether the leader must first recover the previous instances from the completed ballot or can move directly to *Phase 2*.
- `aborted` – tracks whether the new ballot was successfully completed or aborted.

When a new ballot request is received, the server updates both its current and last completed ballot, sets the `activated` and `hasPermissionForPhaseTwo` flags to `false`, and resets the `aborted` flag. It then enters the `waitForCompletedBallot` procedure, where it suspends execution until the outcome of the completed ballot is known. The `updateCompletedBallot` and `abortCompletedBallot` methods are responsible for signaling this procedure, indicating whether the ballot was successfully completed or aborted.

Meanwhile, in the *MainLoop*, the leader waits for console activation before performing any *Phase 2* operations, as implemented in the `checkPermissionForPhaseTwo()` function.

Upon receiving the `NewBallotReply`, the console inspects the `accepted` field to determine whether to grant permission to the new leader. It also tracks the highest completed ballot. When the reply arrives, the console checks the reply's ballot number and the accepted value, updating its own completed ballot and granting permission for phase two only if the new leader's ballot is higher than its own and if *Phase 1* was not aborted in the new leader's process. It gives/blocks permission sending an `ActivateRequest` with the corresponding value (`isMostRecentLeader`) for the leader in question.

When the server wakes up from the `checkPermissionForPhaseTwo()` function (after receiving the console activation), it checks if it can proceed to *Phase 2* or if it needs to break out of the leader zone.

This approach ensures that the new leader does not propose commands until previous ballots have been fully reconciled, enabling safe reconfiguration.

2.6. Fast TOPIC

To improve performance and reduce unnecessary overhead, our implementation introduces the Fast TOPIC mechanism, which allows TOPIC operations to bypass the Paxos consensus protocol. This optimization is designed to preserve causal consistency in a client while enabling faster execution of TOPIC commands that do not affect global agreement.

2.6.1 Motivation and Constraints

In a standard Paxos-based system, all operations are executed via consensus to ensure a consistent replicated log across servers as we described. However, TOPIC operations do not require consensus to preserve safety. The challenge is to ensure that these operations still respect the client's view of the system, avoiding inconsistencies due to out of order execution.

The Fast TOPIC optimization enables the server to execute topic operations locally and immediately, provided that the server has already applied all relevant prior operations that the client has seen to not break causality.

2.6.2 Tracking Causal Progress

To ensure that a TOPIC operation reflects all relevant prior updates, the client includes with each TOPIC request the highest Paxos instance it has seen applied, referred to as `last_instance_number_seen`.

This instance number is tracked by the client and is updated upon receiving replies to certain state-modifying operations that may affect TOPIC operations result, namely,

`openMeeting` and `addParticipant`. These are the only operations in the system that can influence the result of a TOPIC query (e.g., who is currently in a meeting). As such, only these operations return the corresponding Paxos instance number (specifically, `nextToApply - 1`, reflecting the last applied instance at the time of execution).

Operations like `closeMeeting`, which do not affect TOPIC outcomes, do not return this instance number, an intentional optimization to avoid unnecessary coordination, since their omission does not compromise causal correctness.

2.6.3 Server-Side Synchronization

When a client issues a TOPIC operation, the server first checks whether it has already applied all Paxos instances up to and including the one provided in `last_instance_number_seen`. If so, the TOPIC operation is executed immediately. If not, the server must wait until its internal state catches up with the last state client has seen when making the TOPIC request. This makes sure we don't break clients causal consistency.

This check is performed using a coordination mechanism we introduced: `ApplyProgressLock`. This component enables TOPIC-executing threads to wait until the `ApplyThread` has reached the required instance. Once new instances are applied, the `ApplyThread` notifies waiting threads, which then re-check if the required state has been reached.

This design guarantees that Fast TOPIC operations are only executed when the server's state is consistent with the client's view, ensuring that no causality is broken even though consensus is bypassed.

2.7. Summary

Overall, our design incrementally extends the baseline Paxos implementation to support Multi-Paxos, Vertical Paxos (VP2) semantics and blue operations. Through the introduction of threaded execution for *Phase 2*, structured synchronization between the leader and the console, and the decoupling of command application from consensus, our implementation achieves a clear separation of responsibilities and maintains consistency across replicas. This layered approach allows the system to efficiently handle reconfiguration and concurrent command proposals while preserving correctness.

References

[1] L. Lamport. Paxos made simple. *ACM SIGACT News*, 2001.

[2] L. Lamport, D. Malkhi, and L. Zhou. Vertical paxos and primary-backup replication. Technical report, Microsoft Research, Feb. 2009.